

DATABASE SYSTEMS

CHAP. 5: Introduction to SQL

SQL

- Structured Query Language was designed by IBM to create a standardize structure for interrogating and updating databases.
- Now a very widely used standard and, even companies that offer proprietary database access methods normally implement SQL as an option.
- SQL is a very-high-level language: Say “what to do” rather than “how to do it.”
- Avoid a lot of data-manipulation details needed in procedural languages like C++ or Java.

SQL Data types (from Oracle 9i)

- **String types**

- CHAR(n) – fixed-length character data, n characters
long Maximum length = 2000 bytes
- VARCHAR2(n) – variable length character data,
maximum 4000 bytes
- LONG – variable-length character data, up to 4GB.
- Maximum 1 per table

SQL Data types (from Oracle 9i)

- **Numeric types**

- NUMBER(p,q) – general purpose numeric data type
- INTEGER(p) – signed integer, p digits wide
- FLOAT(p) – floating point in scientific notation with p binary digits precision

- **Date/time type**

- DATE – fixed-length date/time in dd-mm-yy form

Data Definition Language (DDL)

- Allows to describe and name entities, attributes, and relationships required for the application and any associated integrity and security constraints.
- Commands that define a database, including creating, altering, and dropping tables and establishing constraints

DDL

- **Create a table**

- CREATE TABLE [table name] (columnname datatype, ...);

- **Example:**

- CREATE TABLE customers (col1 NUMBER, col2 VARCHAR2(20));

DDL

- **Rename a table**

- ALTER TABLE [table name] RENAME TO [new table name];

- **Example:**

- ALTER TABLE customers RENAME TO customer;

DDL

- **Add a column**

- `ALTER TABLE [table name] ADD ( [column name] [datatype], ... );`

- **Example:**

- `ALTER TABLE employee ADD (id int);`

DDL

- **Modify a column**

- ALTER TABLE [table name] MODIFY ([column] [new data type]);

- **Example:**

- ALTER TABLE employee MODIFY(sickHours float);

DDL

- **Drop a column**

- ALTER TABLE [table name] DROP COLUMN [column name];

- **Example:**

- ALTER TABLE employee DROP COLUMN vacationPay;

- **Removing Tables**

- DROP TABLE statement allows you to remove tables from your schema:
 - DROP TABLE employee

Data Manipulation Language (DML)

- Provides basic data manipulation operations on data held in the database.
- Commands that maintain and query a database

DML

- **SELECT FROM WHERE**

**SELECT [ALL / DISTINCT] *expr1* [AS *col1*], *expr2* [AS *col2*]
FROM *tablename*
WHERE *condition***

- **Example:**
SELECT name, hcode, class
FROM student
WHERE class="1A"

DML

- **Grouping**

- **SELECT FROM WHERE *condition* ;**
- **GROUP BY *groupexpr* [HAVING *requirement*]**

- **Group functions**

- **COUNT()**
- **SUM()**
- **AVG()**
- **MAX()**
- **MIN()**

DML

- **Sorting**

**SELECT FROM WHERE
GROUP BY ;**

ORDER BY *colname* ASC / DESC

- **Example:**

- **List the boys of class 1A, order by their names.**

***SELECT name, id
FROM student
WHERE sex="M"
AND class="1A"
ORDER BY name***

DML

- **Output**

- **INTO TABLE tablename:** output table is saved as a database file in the disk.
- **INTO CURSOR temp:** output is stored in the working memory temporarily.
- **TO FILE filename [ADDITIVE]:** output to a text file. (additive = append)
- **TO PRINTER:** send to printer.
- **TO SCREEN:** display on screen.

DML

- Example:
- List the students in desc. order of their names and save the result as a database file name.dbf.

```
SELECT *  
FROM student  
ORDER BY name DESC  
INTO TABLE name.dbf
```

DML

- **Union**

```
SELECT ..... FROM ..... WHERE ..... ;  
UNION ;  
SELECT ..... FROM ..... WHERE .....
```

- **Example:**

- The two clubs want to hold a joint party. Make a list of all students.

```
SELECT *  
FROM bridge  
UNION  
SELECT *  
FROM chess  
ORDER BY class, name  
INTO TABLE party
```

DML

- **Intersection**

```
SELECT ..... FROM table1 ;  
WHERE col IN ( SELECT col FROM table2 )
```

- Example:

- Print a list of students who are members of both clubs. (Intersection)

```
SELECT *  
FROM bridge  
WHERE id IN ( SELECT id FROM chess )  
TO PRINTER
```

DML

- **Difference**

```
SELECT ..... FROM table1 ;  
WHERE col NOT IN ( SELECT col FROM table2 )
```

- **Example:**

- Make a list of students who are members of the Bridge Club but not Chess Club.

```
SELECT *  
FROM bridge  
WHERE id NOT IN ( SELECT id FROM chess )  
INTO TABLE diff
```

Multiple Tables

- SQL provides a convenient operation to retrieve information from multiple tables.
- This operation is called join.
- The join operation will combine the tables into one large table with all possible combinations and then it will filter the rows of this combined table to yield useful information.

Natural Join

- Joins two tables by their common column.
- This operation is similar to the set intersection of two tables.
- Syntax:
SELECT *a.comcol, a.col1, b.col2, expr1, expr2*
FROM *table1 a, table2 b*
WHERE *a.comcol = b.comcol*

Natural Join

- Example:
- Find the number of students learning piano in each class.

```
SELECT s.class, COUNT(*) ;  
FROM student s, music m  
WHERE s.id=m.id  
AND m.type="Piano"  
GROUP BY class  
ORDER BY class
```

Outer Join

- Includes rows that have a match, plus rows that do not have a match in the other table.

Outer Join

- Example:
- Make a checking list of students and the instruments they learn. The list should also contain the students without an instrument.

```
SELECT s.class, s.name, s.id, m.type
FROM student s, music m
WHERE s.id=m.id
UNION
SELECT class, name, id, ""
FROM student
WHERE id NOT IN ( SELECT id FROM music )
ORDER BY 1, 2
```

Adds data to a table

- INSERT

- Insert a single row into a table:

- INSERT INTO table_name VALUES (col1, col2, ...);

- Inserting a record with all fields

- INSERT INTO CUSTOMER_T VALUES (001, 'Contemporary Casuals', 1355 S. Himes Blvd.', 'Gainesville', 'FL', 32601);

Adds data to a table

- Inserting a record with specified fields
 - `INSERT INTO PRODUCT_T (PRODUCT_ID, PRODUCT_DESCRIPTION, PRODUCT_FINISH, STANDARD_PRICE, PRODUCT_ON_HAND) VALUES (1, 'End Table', 'Cherry', 175, 8);`
- Insert rows from one table into another:
 - `INSERT INTO table_name(col1, col2, ...) (SELECT 'value1', 'value2', ... from table_name);`
 - `INSERT INTO CA_CUSTOMER_T SELECT * FROM CUSTOMER_T WHERE STATE = 'CA';`

Modifies data in existing rows

- UPDATE
 - Update the entire column of that table (all rows):
 - UPDATE customer SET state='CA';
 - Update the specific record of the table:
 - UPDATE customer SET name='Joe' WHERE customer_id=10;
 - Updates the column invoice as paid when paid column has more than zero:
 - UPDATE movies SET invoice='paid' WHERE paid >= 0;

Removes rows from a table

- DELETE
 - Delete all rows from a table:
 - `DELETE FROM tab1;`
 - `DELETE FROM CUSTOMER_T;`
 - Conditionally delete rows (Delete certain rows)
 - `DELETE FROM tab1 WHERE col1 = '123';`
 - `DELETE FROM CUSTOMER_T WHERE STATE = 'HI';`

Control Language (DCL)

- Commands that control a database, including administering privileges and committing data

DCL

- GRANT

- Grant privileges to users or roles:

- GRANT create session TO scott;

- REVOKE

- Revoke privileges from users or roles:

- REVOKE unlimited tablespace FROM scott;

VIEW

- A view is a virtual table defined by an SQL statement.
- Types of Views
 - Dynamic Views
 - Materialized Views

VIEW

- **Example:**

- *CREATE VIEW EXPENSIVE_STUFF_V AS*
- *SELECT PRODUCT_ID, PRODUCT_NAME, UNIT_PRICE*
- *FROM PRODUCT_T*
- *WHERE UNIT_PRICE >300*
- *WITH CHECK_OPTION;*

Advantages of Views

- Simplify query commands
- Assist with data security (but don't rely on views for security, there are more important security measures)
- Enhance programming productivity
- Contain most current base table data
- Use little storage space
- Provide customized view for user
- Establish physical data independence

Disadvantages of Views

- Use processing time each time view is referenced
- May or may not be directly updateable